FormPilot - Build Guide

How to build it, the exact stack and skills, in reliable phases - finishable with Claude Code in 5 to 10 hours.

Scope: a local-first Chrome/Edge extension (Manifest V3). No backend. No accounts. Works offline.

Encrypted document vault + intelligent form autofill + resize-to-portal-spec + OCR extract

A practical, beginner-friendly plan - build one phase per Claude Code session, test, commit, repeat.

1. What you are building (and what you are NOT)

To be finishable in 5-10 hours and genuinely reliable, this build targets one platform: a local-first browser extension for Chrome and Edge (Manifest V3). Everything runs and stores on the user's own machine - no server, no login, works offline. That single decision removes the hardest, slowest parts (backend, auth, hosting, sync) and still delivers the four features that make the product special.

Capability	What it means	Status
Encrypted document vault	Store personal fields + document images locally, encrypted with a passphrase.	In scope
Intelligent form autofill	Detect fields on any web form and fill them from the vault, with review before submit.	In scope
Resize/compress to portal spec	Output a photo/signature/doc at an exact format + dimension + file-size band.	In scope
OCR auto-extract	Read an ID image and pre-fill fields (stretch goal / last phase).	In scope (stretch)
Mobile app, cloud sync, DigiLocker API	Powerful but slow to build and easy to get wrong.	Later (not in 5-10h)

Keeping mobile and cloud out of the first build is what makes it reliable and finishable. You can add them later on the same vault design.

2. The tech stack (deliberately simple)

Simple, boring and well-documented beats clever here - it is easier for you to understand, easier for Claude Code to generate correctly, and less likely to break.

Layer	Choice	Why
Language	Vanilla JavaScript + HTML + CSS	No framework needed; fewer moving parts, nothing to misconfigure.
Platform	Chrome/Edge extension, Manifest V3	The only supported extension format; huge install base; local by nature.
Storage	chrome.storage.local	Built-in, persistent, offline key-value store for the extension.
Encryption	Web Crypto API (AES-256-GCM + PBKDF2)	Native browser crypto; derive a key from the user passphrase; no library needed.
Image resize	browser-image-compression + Canvas toBlob	Target a file-size band by iterating quality/scale (binary search).
OCR	tesseract.js (WASM)	On-device text recognition, packaged inside the extension.
Tooling	Node.js, Git, VS Code	Node runs Claude Code and any optional bundling; Git for commits per phase.
AI builder	Claude Code (Pro plan)	Your pair-programmer: it writes and edits files and runs commands with your approval.

Recommended: the 'no build step' route - download the prebuilt library files (browser-image-compression, tesseract.js) into a /vendor folder and reference them directly. This keeps things beginner-friendly and satisfies Manifest V3's ban on remotely hosted code.

3. Skills you need (most you will pick up as you go)

You do not need all of this up front. With Claude Code writing the code and explaining it, you mainly need to understand enough to review, test and fix. Rough map:

Bucket	What it covers
Comfortable already helps	HTML/CSS/JS basics, running commands in a terminal, Git add/commit.

Learn during the build	Chrome extension model (manifest, service worker, content script, message passing, chrome.storage); DOM/form fields; async/await.
Learn just-in-time (Claude explains)	Web Crypto (AES-GCM, PBKDF2); Canvas API + Blob/File; tesseract.js worker API.
Meta-skill	Working with an AI coding agent: scoped prompts, reviewing diffs, testing after each step.

4. The phased plan at a glance

Build one phase per Claude Code session. After each phase: load the extension, test the 'done when' check, then commit with Git. If a session runs long, stop at a phase boundary - never mid-phase.

#	Phase	Done when...	Time
0	Setup & scaffold	Node + Claude Code installed; empty MV3 extension loads; popup opens	0.5 - 1 h
1	Encrypted vault	Add/edit personal fields + documents; encrypted at rest; passphrase unlock	1.5 - 2 h
2	Autofill engine	'Fill this form' fills a real web form from the vault, with review; never auto-submits	2 - 3 h
3	Resize-to-spec tool	Pick an image + preset -> output lands inside the required KB band and dimensions	1.5 - 2 h
4	OCR auto-extract (stretch)	Upload an ID image -> fields pre-filled from recognised text + confidence	1 - 1.5 h
5	Polish & reliability	Idle lock, encrypted backup export/import, error states, README	~1 h

Total: roughly 7.5 - 10.5 hours. Phases 0-3 are the reliable core (about 5.5-8 h) and already make a compelling, usable tool. Phase 4 is a stretch; Phase 5 makes it feel finished.

5. Phase-by-phase build instructions

Phase 0 - Setup & scaffold (0.5-1 h)

Install Node.js (LTS) and Git. Install Claude Code and run it inside a new project folder. Create the four-file Manifest V3 skeleton and load it unpacked in the browser.

- Install Claude Code (npm package @anthropic-ai/claude-code), then run 'claude' in your project folder. It edits files and runs commands with your approval.
- Files: manifest.json, background.js (service worker), popup.html + popup.js, options.html + options.js, /icons.
- Load it: Chrome -> Extensions -> Developer mode -> Load unpacked -> pick the folder.

Paste to Claude Code:

```
Create a Manifest V3 Chrome extension skeleton in this folder called FormPilot.
Include manifest.json (permissions: storage, activeTab, scripting; host_permissions:
<all_urls>), a background service worker, a popup (popup.html/js) with a title and one
button, and an options page (options.html/js). Keep it vanilla JS, no frameworks. Then
tell me the exact steps to load it unpacked in Chrome and confirm the popup opens.
```

Done when: the extension loads with no errors and the popup opens. Commit: 'phase 0: scaffold'.

Phase 1 - Encrypted document vault (1.5-2 h)

Build the options page into a vault: a form to store personal fields (name, DOB, address, email, phone, PAN, Aadhaar-masked, etc.) plus uploaded document images. Encrypt everything with a key derived from a passphrase using Web Crypto (PBKDF2 -> AES-256-GCM). Nothing is stored in plaintext.

- On first run, user sets a passphrase; derive a key (PBKDF2, high iteration count, random salt).
- Encrypt the vault JSON with AES-GCM (random IV per save) before writing to chrome.storage.local.
- Unlock on open by re-deriving the key; keep the key only in memory for the session.
- Store document images as base64/Blob inside the encrypted vault, tagged by type.

Paste to Claude Code:

```
In the options page, build an encrypted vault. Add a passphrase setup + unlock flow using
the Web Crypto API: derive an AES-256-GCM key with PBKDF2 (random salt, >=150k iterations).
Provide a form to add/edit fields (fullName, dob, email, phone, address, pan, aadhaarMasked,
plus custom fields) and to attach document images by type. Encrypt the whole vault (random IV
per save) and store it in chrome.storage.local. Never store plaintext. Add lock/unlock buttons.
Explain the crypto steps in comments so I can follow them.
```

Done when: you add data, lock, reload, unlock with the passphrase, and see it again; the raw stored value is unreadable ciphertext. Commit: 'phase 1: encrypted vault'.

Phase 2 - Autofill engine (2-3 h) [the core feature]

A content script inspects the current page's form fields and infers each field's meaning; the popup's 'Fill this form' button sends the vault data to the content script, which fills matching fields and highlights them for review. It must never auto-submit.

- Match order: the standard autocomplete attribute first, then name / id / label / placeholder / aria-label against a synonyms dictionary (e.g. 'dob' = 'date of birth' = 'birth date').
- Popup -> content script via chrome.tabs.sendMessage; content script fills inputs and fires input/change events so sites register the value.
- Highlight filled fields; show a small count ('filled 7 of 9'); leave unknown fields for the user.
- Safety: never call form.submit(); block auto-fill of password fields; optionally warn on low-trust domains.
- Reliability booster: let the user save a correction as a per-site mapping so the next visit is perfect.

Paste to Claude Code:

```
Add autofill. Write a content script that scans the active page for input/select/textarea
fields and infers each field's purpose using, in order: the autocomplete attribute, then
name/id/label/placeholder/aria-label matched against a synonyms map I can extend. Add a
'Fill this form' button in the popup that sends the unlocked vault data to the content script,
which fills matching fields, dispatches input and change events, and outlines filled fields.
Never auto-submit and never fill password fields. Show 'filled X of Y'. Let me save a
per-site field mapping correction to chrome.storage for next time.
```

Done when: on a real signup or application form, clicking 'Fill this form' correctly fills your stored fields, highlights them, and does not submit. Commit: 'phase 2: autofill'.

Phase 3 - Resize/compress to portal spec (1.5-2 h) [the differentiator]

A vault tool where the user picks an image and a preset describing {format, max dimension, min-max file-size KB}. Using browser-image-compression plus a Canvas toBlob binary search on quality, the output is nudged until it lands inside the required file-size band, previewed, then saved or downloaded for the form's file input.

- Ship 3-4 presets (e.g. 'Photo: JPEG, 3.5x4.5 cm, 10-200 KB', 'Signature: JPEG, 4-30 KB') and allow custom presets.
- Resize longest edge to the max dimension first, then binary-search JPEG quality to hit the KB band.
- Warn if the target is impossible (too small a band) and show original vs result size + percent saved.
- Save the result back into the vault, and offer download so it can be attached to the form.

Paste to Claude Code:

```
Add an image tool to the options page. Vendor browser-image-compression into /vendor and load
it locally (no CDN, MV3-safe). Let the user pick an image and a preset defined as
{format, maxWidthOrHeight, minKB, maxKB}. Resize to the max dimension, then binary-search the
JPEG quality with canvas.toBlob until the output size is within [minKB, maxKB]. Show original vs
compressed size and percent saved, preview the result, and add Save-to-vault and Download
buttons. Include 3 presets plus a custom-preset form. Handle the impossible-band case gracefully.
```

Done when: you feed a big photo + a '10-200 KB JPEG' preset and get a file inside that band at the right dimensions. Commit: 'phase 3: resize-to-spec'.

Phase 4 - OCR auto-extract (1-1.5 h) [stretch]

Add tesseract.js (packaged locally) so the user can drop an ID image and have fields pre-filled from the recognised text using simple patterns (PAN format, dates, name lines), with a confidence readout and easy manual correction.

- Vendor tesseract.js core + worker + English traineddata into /vendor; set corePath/workerPath/langPath to those local files (MV3 forbids remote code).
- Run recognition in its worker; show the progress logger; read data.text and data.confidence.
- Extract with light regex/heuristics (e.g. PAN = 5 letters + 4 digits + 1 letter; DOB patterns) into the vault fields for review.
- Always let the user correct misreads; warn when confidence is low.

Paste to Claude Code:

```
Add OCR with tesseract.js, vendored into /vendor and configured to load its core, worker and
eng traineddata from local files (no CDN, MV3-safe). Let the user pick an ID image; run
recognition in the worker with a progress indicator; then pre-fill vault fields using simple
regex heuristics (PAN pattern, date-of-birth, name line). Show the OCR confidence and let me
edit every field before saving. Keep it fully on-device.
```

Done when: a clear PAN/marksheet image pre-fills at least a couple of fields you can then correct. Commit: 'phase 4: ocr'. If time is short, skip this and keep it as a next step.

Phase 5 - Polish & reliability (~1 h)

Make it feel finished and trustworthy.

- Auto-lock the vault after a few minutes idle (clear the in-memory key).
- Encrypted backup: export the vault to an encrypted file and import it back (so data survives a reinstall).
- Empty states, clear error messages, a confirm step before overwriting data.
- A README with setup, features, and a one-line privacy statement ('all data stays on your device').

Paste to Claude Code:

```
Add: idle auto-lock (clear the key after N minutes), encrypted export/import of the vault to a
file, friendly empty and error states, and a README documenting setup, features and privacy.
Do a quick self-review for anything that could store plaintext or auto-submit a form, and fix it.
```

Done when: lock-on-idle works, and you can export, reinstall, import, and recover your vault. Commit: 'phase 5: polish'.

6. Making it reliable and useful for many purposes

- **Match on standards first.** The HTML autocomplete attribute is the most reliable signal; fall back to a synonyms dictionary for name/id/label. This is what lets one tool fill many different sites.
- **Let users teach it.** Saving per-site field mappings turns a 70%-right guess into 100% on the next visit - the single biggest reliability win.
- **Never surprise the user.** Review-before-submit, no auto-submit, no filling passwords, and a visible 'filled X of Y' build trust.
- **Encrypt everything at rest and lock on idle.** Local does not mean unprotected.
- **Ship an encrypted backup.** Data that can vanish on reinstall will not be trusted; export/import fixes that.
- **Broad, editable schema.** Support custom fields and many document types so it covers exams, KYC, jobs, visas, insurance - not just one form.
- **Fail gracefully.** Show OCR confidence, warn on impossible image targets, and always allow manual correction.

7. Known gotchas (save yourself hours)

Gotcha	Fix
Manifest V3 bans remote code	OCR/compression libraries loaded from a CDN will be rejected. Vendor them into /vendor and point paths (corePath/workerPath/langPath) at local files.
Service worker is ephemeral	It sleeps when idle, wiping globals. Keep all state in chrome.storage, not in variables.
Separate execution contexts	Popup, options, content script and service worker cannot call each other directly - use chrome.tabs.sendMessage / runtime messaging.

Sites ignore programmatic fills	After setting a field's value, dispatch 'input' and 'change' events or React/Vue sites won't register it.
tesseract.js first-run is slow	The WASM core and language file load once; show a progress bar and cache locally.
Image band can be unreachable	A 4-30 KB target on a huge photo may be impossible at min quality - resize dimensions first and warn the user.

8. Getting the most out of Claude Code (Pro)

- **Keep two files at the repo root.** CLAUDE.md (project rules: vanilla JS, MV3, local-only, never auto-submit, never store plaintext) and PLAN.md (this phase list). Claude Code reads them for context.
- **One phase per session.** Use the prompt boxes above. Scoped prompts produce correct, reviewable changes; giant prompts drift.
- **Test, then commit, after every phase.** 'git commit' at each boundary gives you a safe point to roll back to.
- **Review diffs; ask for explanations.** 'Explain the crypto in plain English' or 'why dispatch change events?' - understanding beats copy-paste.
- **If it over-builds, say 'smaller steps'.** Ask it to load the extension and self-test, and to fix any console errors.
- **Mind Pro usage limits.** Pro includes Claude Code with usage that resets on a rolling window; heavier phases (autofill, OCR) are best started with fresh capacity. Keep sessions scoped so a limit never interrupts you mid-phase.
- **Docs:** Claude Code - docs.claude.com/en/docs/claude-code/overview.

9. Suggested repo structure

```
FormPilot/
  manifest.json
  background.js
  popup.html  popup.js
  options.html  options.js  (vault + image tool + OCR)
  content.js  (field detection + fill)
  lib/  crypto.js  match.js  image.js  ocr.js
  vendor/  browser-image-compression.js  tesseract/ (core, worker, eng.traineddata)
  icons/  CLAUDE.md  PLAN.md  README.md
```

10. Definition of done (v1)

- Loads as an unpacked MV3 extension with no console errors.
- Vault stores fields + document images, encrypted; unlocks by passphrase; auto-locks on idle.
- 'Fill this form' correctly fills a real web form with review and no auto-submit.
- Image tool outputs a file inside a chosen KB band at the right dimensions.
- (Stretch) OCR pre-fills a couple of fields from an ID image with confidence shown.
- Encrypted export/import works; README present; everything runs offline.

Prepared as a build guide. Library APIs and Manifest V3 / Claude Code details evolve - check the official docs (each tool's site and docs.claude.com) before and during the build. Follow UIDAI norms for any real Aadhaar handling: mask by default and store the minimum.